Studio Nx

Backend(FastAPI/Python) 32 — POST /api/v1/auth/login

2026-10-12

Backend(FastAPI/Python) 32 — 라우터 `POST /api/v1/auth/login`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `login`)
작업	P1-05 로그인: 로그인 API (PRD 01 FR-103, FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	31 리프레시 쿠키 · 33 `POST /auth/refresh`

1. 하는 일

이메일과 비밀번호를 확인하고, **토큰 두 개**를 서로 다른 경로로 나눠서 줍니다.

```
POST /api/v1/auth/login
{"email": "route.user@example.com", "password": "Abcdef1!", "remember": true}

→ 200
Set-Cookie: snx_refresh=eyJ…(3일); HttpOnly; Max-Age=259200; Path=/api/v1/auth; SameSite=lax
{"access_token": "eyJ…(30분)", "token_type": "bearer", "expires_in": 1800}
```

토큰	전달되는 곳	만드는 함수
Access (30분)	응답 **본문**으로 가서 JS 메모리에 보관됩니다	26
Refresh (3일)	응답 **쿠키**(`HttpOnly`)로 가며, JS는 볼 수 없습니다	27, 31

경우	응답
맞음	200과 위의 내용
비밀번호 틀림, 없는 이메일, 탈퇴 회원	401 `INVALID_CREDENTIALS` "Email or password is incorrect."이고, **쿠키는 없습니다**
정지된 회원 (비밀번호는 맞음)	403 `ACCOUNT_DISABLED`
칸이 빠짐	422

2. 언제 쓰나

home.html 로그인 모달의 **Sign In** 버튼을 누르면 `home.js`와 `auth-ui.js`(P1-07)가 이 API를 부릅니다.

순서	화면 또는 서버	일어나는 일
①	로그인 모달	이메일, 비밀번호, **Keep me signed in**을 입력합니다
②	P1-07 JS	`api.post('/auth/login', {email, password, remember}, {auth: false})`를 보냅니다
③	**이 라우터**	29번으로 확인하고, 27번과 31번으로 쿠키를 만든 뒤, 26번으로 본문을 만듭니다
④	P1-07 JS	`setAccessToken(access_token)`을 부르고(JS 01), 헤더를 My Page · My Projects로 바꾼 뒤 `next` 페이지로 이동합니다 (JS 05)

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.core.security import create_access_token, create_refresh_token
from app.schemas.auth import (
    AvailabilityResponse,
    LoginRequest,
    Nickname,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`from app.core.security import create_access_token, create_refresh_token`을 이번에 추가했습니다.** 이 함수는 `Response`, `Db`, `get_settings`, 두 토큰 함수, `LoginRequest`, `TokenResponse`, `auth_service`를 사용합니다.

함수

```
@router.post(
    "/login",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}, 403: {"model": ErrorResponse}},
)
def login(data: LoginRequest, response: Response, db: Db) -> TokenResponse:
    user = auth_service.authenticate(db, data.email, data.password)
    set_refresh_cookie(response, create_refresh_token(user.user_id), data.remember)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

`@router.post(…)`와 함수 머리는 템플릿에 있었고, 몸통 세 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from app.core.security import A, B`	26번과 27번의 토큰 함수 두 개를 가져옵니다

식	뜻
`response: Response`	FastAPI가 넣어 주는 응답 객체입니다. 여기에 쿠키를 붙이면(31) 최종 응답 헤더에 실립니다

식	뜻
`data: LoginRequest`	본문 `{email, password, remember}` 입니다 (`remember`가 없으면 `False`)
`TokenResponse(access_token=…, expires_in=1800)`	응답 본문입니다. `token_type`은 기본값 `"bearer"` 입니다

4. 흐름

1. 29번 `authenticate()`로 확인합니다. 틀리면 여기서 401이나 403으로 끝납니다 (쿠키도 토큰도 만들지 않습니다).
2. 27번으로 리프레시 토큰을 만들고, 31번으로 쿠키에 담습니다 (`remember`에 따라 3일 쿠키 또는 세션 쿠키).
3. 26번으로 Access Token을 만들어 본문에 담고 200으로 응답합니다.

5. 왜 이렇게

선택	이유
확인을 맨 먼저 함	실패하면 토큰도 쿠키도 만들지 않습니다. 시험에서 실패 응답에 `Set-Cookie`가 없다는 것을 확인했습니다
Access는 본문으로, Refresh는 쿠키로	27 §5에서 설명했듯이, 짧은 토큰은 JS가 쓰고 긴 토큰은 JS가 볼 수 없게 숨깁니다
`expires_in`을 함께 줌	화면이 "몇 초 뒤에 만료되는지" 알 수 있습니다. 지금 `api.js`는 401을 받으면 재발급하는 방식이라 쓰지 않지만, 만료 전에 미리 재발급하는 방식으로 바꿀 때 쓸 수 있습니다
이메일 형식 검사 (`EmailStr`)를 하지 않음	로그인 칸에 형식이 틀린 이메일을 넣어도 422가 아니라 401 "Email or password is incorrect." 하나로 답합니다. 무엇이 틀렸는지 알려 주지 않는 원칙(29)과 같습니다

6. 시험 (`tests/test_auth_routes.py`에 7개를 추가해서 모두 34개, 실제 앱과 MySQL로 실행)

가입(22)으로 회원을 만든 뒤 로그인했습니다.

시험	확인한 것
200 본문	칸이 `access_token`, `token_type`, `expires_in` 세 개뿐이고(리프레시 토큰은 없음), `bearer`와 1800이며, 05번으로 읽으면 그 회원 번호가 나옵니다
쿠키 (체크함, 체크하지 않음)	`snx_refresh=…; HttpOnly; Path=/api/v1/auth`이고, 체크했으면 `Max-Age=259200`, 체크하지 않았으면 없습니다. 쿠키 값을 28번으로 읽으면 그 회원 번호가 나옵니다
마지막 로그인	`last_login_at`이 기록됩니다
비밀번호 틀림, 없는 이메일	둘 다 401이고 본문이 같으며, `Set-Cookie`가 없습니다
비밀번호 칸이 빠짐	422

전체 시험 225개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.